

Deep Learning for Physicists: A Physics-Education Review of Function Approximation and Scientific Generalization

Othman H. Y. Zalloum  Department of Applied Mathematics and Physics, Palestine Polytechnic University, Hebron, Palestine **E-mail:** ozalloum@ppu.edu

Abstract

This review introduces deep learning to advanced undergraduate and beginning graduate physics students through a physics-first account of function approximation, optimization, automatic differentiation, and scientific validation. Its central educational question is what a reported test error establishes when observations are generated by physical systems rather than by exchangeable abstract samples. A reproducible damped-oscillator benchmark distinguishes random-point, system-level, parameter-interpolation, and parameter-extrapolation tests. Across three independent initializations, random-point validation MSE is $(1.98e-03 \pm 1.04e-03)$, whereas evaluation on entirely unseen trajectories gives $(8.39e-03 \pm 2.33e-03)$; mean relative L^2 error is 0.062 ± 0.014 for interpolation and 0.273 ± 0.028 for extrapolation. A second analytically controlled simple-pendulum example crosses the energy separatrix between bounded oscillation and continuous rotation, with error increasing from 0.079 ± 0.006 on held-out oscillatory systems to 1.080 ± 0.027 in the rotation regime. Conventional baselines, a compact PyTorch implementation, and a small differentiable inverse problem connect the concepts to computational practice. The review concludes with learning outcomes and student activities that help learners distinguish interpolation from extrapolation, select physically meaningful validation units, and judge a surrogate against appropriate numerical and non-neural alternatives. It does not claim a new neural architecture; its contribution is a reproducible educational framework for interpreting scientific generalization.

Keywords: physics education, deep learning, computational physics, scientific machine learning, generalization, surrogate modeling, inverse problems

1 Educational purpose and scope

1.1 Audience and learning outcomes

This review is designed for an upper-level undergraduate or beginning graduate course in computational physics, numerical methods, or scientific machine learning. It assumes linear algebra, multivariable calculus, ordinary differential equations, and basic numerical computation, but no previous machine-learning course. The mathematics is kept explicit while the examples remain small enough to reproduce in a teaching laboratory or independent student project.

After studying the review and running the accompanying notebook, students should be able to:

1. interpret a feed-forward neural network as a parameterized nonlinear approximation and relate its training to numerical optimization;
2. explain the role of back propagation and automatic differentiation in fitting a model and computing sensitivities;
3. define the physical prediction unit before constructing a training-validation split;
4. distinguish sample-level validation from system-level validation, parameter interpolation, parameter extrapolation, and qualitative regime-shift tests; and
5. compare a learned surrogate with a direct solver and a conventional data-driven baseline before making a scientific claim.

The accompanying code is therefore part of the teaching design rather than only a computational supplement. An instructor can use the oscillator example for a guided laboratory, ask students to

Table 1. Closest prior work and the deliberately narrower role of the present article.

Work	Principal emphasis	Distinction of the present treatment
Higham & Higham (2019)[1]	First-principles mathematical introduction to deep learning	Physical systems replace generic examples; validation by system, parameter, and regime is central.
Mehta <i>et al.</i> (2019)[2]	Broad machine-learning introduction for physicists with notebooks	Restricts the method space to feed-forward regression so the scientific meaning of a test split can be developed in depth.
Carleo <i>et al.</i> (2019)[3]	Review of ML across the physical sciences	Tutorial and benchmark-driven rather than an applications survey.
Ray, Pinti & Oberai (2024)[4]	Textbook linking deep learning and computational physics	Article-length progression organized around a validation hierarchy rather than an architecture curriculum.
Rabeh <i>et al.</i> (2025)[17]	SciML benchmarking with random and extrapolatory CFD splits	Uses the distribution-shift idea pedagogically from the first neural network and separates sample, system, parameter, and qualitative regime shifts.
Chu <i>et al.</i> (2026)[19]	Foundation-model benchmark across physical regimes and distribution shifts	Studies small transparent networks and analytically controlled systems so each validation failure can be derived and reproduced by students.

change the split while holding the architecture fixed, and use the simple-pendulum example to discuss how a qualitative physical boundary differs from a large but smooth parameter displacement. The review is not presented as a controlled intervention study in physics education; its educational contribution is a scholarly, reproducible introduction that connects contemporary scientific machine learning with established computational-physics reasoning.

Deep learning has become part of the computational vocabulary of physics. Neural networks are used as surrogate models, feature extractors, emulators, inverse maps, and components of scientific workflows ranging from particle physics and materials science to fluid mechanics and imaging. Broad reviews have documented this expansion and its connections with physical science.[3, 2] The resulting literature is large enough that a newcomer can now face the opposite problem from a decade ago: there is too much to read before the basic computational ideas become clear.

This review has a deliberately narrow aim. We restrict attention to fully connected feed-forward networks and use a single family of damped oscillators as the main running example. The goal is not to survey the modern deep-learning ecosystem. It is to make a small set of ideas precise enough that a physics student can implement them, criticize them, and recognize when they are being used outside the conditions under which they were validated.

This choice is also intended to distinguish the present treatment from several important predecessors. Higham and Higham gave a concise mathematical introduction to deep learning built around optimization and back propagation;[1] Mehta *et al.* provided a much broader machine-learning introduction for physicists with Jupyter notebooks and physics-inspired data;[2] Carleo *et al.* surveyed machine learning across the physical sciences;[3] and Ray, Pinti, and Oberai developed a textbook treatment explicitly linking deep learning with computational physics.[4] More recent pedagogical work has also introduced neural networks through progressively richer physical examples.[5] At the same time, modern scientific-ML benchmarks explicitly study extrapolatory and distribution-shift evaluation, including flow problems with random versus extrapolatory splits[17] and recent regime-aware foundation-model benchmarks.[19] The present article therefore does *not* claim that either “deep learning for physicists” or out-of-distribution testing is new. Its contribution is a compact teaching framework organized around physically meaningful validation, demonstrated on two analytically controlled systems, with initialization sensitivity and conventional numerical and non-neural baselines. Table 1 summarizes these distinctions.

Contribution and claim boundary. The contribution is methodological, pedagogical, and empirical rather than architectural. Specifically, the article (i) develops an operational validation hierarchy that separates sample-, system-, parameter-, and qualitative-regime generalization; (ii) applies that hierarchy to two analytically controlled physical systems while holding the learning machinery intentionally simple; and (iii) reports initialization sensitivity together with non-neural and direct numerical baselines so that neural accuracy is interpreted relative to conventional computational tools. We do not claim novelty for feed-forward neural networks, out-of-distribution

evaluation, surrogate modeling, or regime-shift testing individually. The proposed contribution is the controlled physics-first organization of these ingredients into a reproducible introductory framework in which the validation protocol itself is treated as part of the scientific question.

The article is organized around four questions. First, what mathematical object does a feed-forward neural network represent? Second, how are its parameters fitted? Third, what should “unseen data” mean when samples are generated by physical systems? Fourth, when is a learned surrogate preferable to a conventional numerical calculation? These questions lead naturally to a small inverse problem at the end. Physics-informed neural networks, neural operators, equivariant architectures, and foundation models are intentionally deferred; they require their own assumptions and validation questions.

2 Start with the physics: a damped oscillator

Consider the underdamped oscillator

$$m\ddot{x} + c\dot{x} + kx = 0, \quad (1)$$

with $x(0) = 1$ and $\dot{x}(0) = 0$. It is convenient to introduce the natural frequency and damping ratio

$$\omega_0 = \sqrt{k/m}, \quad \zeta = \frac{c}{2\sqrt{km}}, \quad (2)$$

so that, for $0 < \zeta < 1$,

$$x(t; \omega_0, \zeta) = e^{-\zeta\omega_0 t} \left[\cos(\omega_d t) + \frac{\zeta}{\sqrt{1 - \zeta^2}} \sin(\omega_d t) \right], \quad \omega_d = \omega_0 \sqrt{1 - \zeta^2}. \quad (3)$$

We will ask a neural network to approximate the map

$$(t, \omega_0, \zeta) \mapsto x(t; \omega_0, \zeta). \quad (4)$$

This example is intentionally simple. The exact solution is available, so approximation error can be measured without ambiguity; the parameters have clear physical interpretations; and changing (ω_0, ζ) generates genuinely different systems rather than neighboring measurements from one system. Those properties make it useful for discussing validation.

Our reference dataset contains 60 training trajectories with $\omega_0 \in [0.8, 2.0]$, $\zeta \in [0.03, 0.35]$, and 60 sampled times on $0 \leq t \leq 10$. A separate set of 15 parameter pairs is reserved for trajectory-level validation. The precise random seed, generated CSV files, model state, and training history are included with the reproducibility package.

3 Neural networks as nonlinear approximations

A fully connected neural network is a parameterized function; standard deep-learning texts develop this construction in much greater breadth.[6] For an input vector $\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{n_0}$, a hidden layer computes

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad (5)$$

$$\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \quad \ell = 1, \dots, L-1, \quad (6)$$

followed by an output map. The trainable parameters are the entries of all weight matrices $W^{(\ell)}$ and bias vectors $\mathbf{b}^{(\ell)}$. In our oscillator example the input is three-dimensional, $\mathbf{x} = (t, \omega_0, \zeta)$, and the output is the scalar displacement $\hat{x} = f_\theta(t, \omega_0, \zeta)$.

Classical universal-approximation results show that sufficiently rich neural networks can approximate broad classes of functions.[7, 8] This fact is important but easy to overinterpret. It is an existence statement, not a guarantee that a finite network trained from finite data will find a good approximation, extrapolate beyond the training distribution, or respect a physical law. The practical questions concern representation, optimization, data coverage, and evaluation.

We use three hidden layers of width 48 with hyperbolic-tangent activations. Before entering the network, each input is affinely scaled to approximately $[-1, 1]$. Scaling is not cosmetic: parameters measured in seconds, radians per second, and dimensionless damping can otherwise enter optimization with very different numerical magnitudes.

A neural network as a parameterized nonlinear approximation

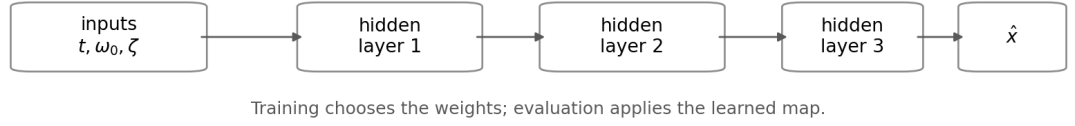


Figure 1. The feed-forward network used throughout the article. Thinking of the network as a parameterized nonlinear approximation is often more useful to a physicist than treating it as an opaque “AI model.”

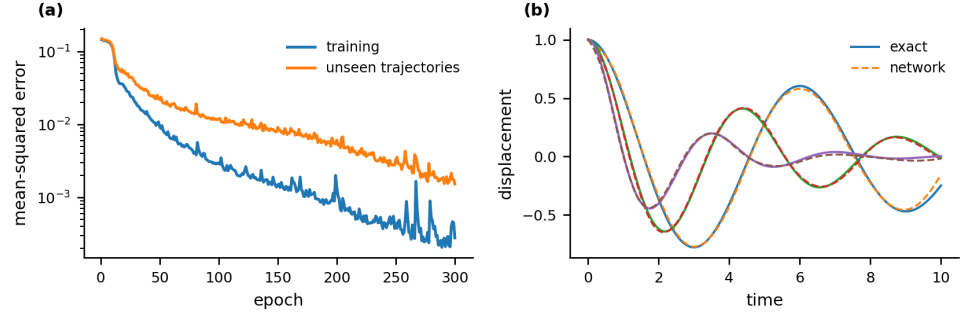


Figure 2. (a) Training and trajectory-level validation losses. (b) Representative predictions for parameter combinations inside the training range. The point of the example is not that a small network can fit a damped oscillator; it is that the validation question can be changed while holding the physical problem fixed.

4 Training is an optimization problem

Given observations $(\mathbf{x}_i, y_i)$, the network parameters θ are chosen to reduce an objective function. For regression, the mean-squared error is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N [f_{\theta}(\mathbf{x}_i) - y_i]^2. \quad (7)$$

Training is therefore a high-dimensional numerical optimization problem. A basic gradient step is

$$\theta_{k+1} = \theta_k - \eta_k \nabla_{\theta} \mathcal{L}(\theta_k), \quad (8)$$

where η_k is the learning rate. In large datasets the gradient is commonly estimated on a mini-batch rather than on every observation. Modern optimizers such as Adam adapt the step using running estimates of first and second moments of the gradient.[11] The language of machine learning should not obscure the numerical-analysis perspective: learning-rate choice, conditioning, scaling, initialization, and stopping criteria all affect the computation.[10]

The reference model is trained for 300 epochs with Adam and mini-batches of 256. Figure 2a shows both training loss and the loss on trajectories whose parameter pairs were not used for fitting. The final training MSE is 2.770e-04 and the trajectory-level validation MSE is 1.521e-03.

5 Back propagation and automatic differentiation

The gradient in Eq. (7) contains derivatives with respect to every weight and bias. Back propagation is the efficient organization of the chain rule that reuses intermediate sensitivities rather than differentiating each parameter independently.[9] For a scalar loss $\mathcal{L}$ and layer preactivation $\mathbf{z}^{(\ell)}$, define

$$\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}. \quad (9)$$

Then the backward recursion has the schematic form

$$\delta^{(\ell)} = \left(W^{(\ell+1)} \right)^T \delta^{(\ell+1)} \odot \sigma' \left(\mathbf{z}^{(\ell)} \right), \quad (10)$$

and

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(\mathbf{a}^{(\ell-1)} \right)^T. \quad (11)$$

In contemporary software one normally does not write these derivatives by hand. Automatic differentiation records the elementary operations in a computational graph and evaluates derivatives by systematic application of the chain rule.[12] Back propagation is therefore best viewed as the reverse-mode differentiation strategy that makes gradient-based training computationally feasible, while automatic-differentiation software is the mechanism that executes it.

For physicists, this viewpoint has an important consequence. Once f_θ is differentiable with respect to its inputs and parameters, the same machinery can be used not only to train the network but also to compute sensitivities of predictions with respect to physical parameters. That observation will support the inverse example in Sec. 11 and, in more advanced settings, leads toward physics-informed learning and differentiable simulation.

6 A complete PyTorch training loop

The central operations fit in a short program. PyTorch[13] handles tensor operations and reverse-mode differentiation; the remaining logic is ordinary numerical programming.

Listing 1. Core training loop for the oscillator surrogate. The full reproducible script writes every dataset, metric, and figure used in this article.

```
model = MLP(width=48, depth=3)
optimizer = torch.optim.Adam(model.parameters(), lr=2e-3)
loss_fn = torch.nn.MSELoss()

for epoch in range(300):
    permutation = torch.randperm(len(X_train))
    for i in range(0, len(X_train), 256):
        idx = permutation[i:i+256]
        prediction = model(X_train[idx])
        loss = loss_fn(prediction, y_train[idx])

        optimizer.zero_grad()
        loss.backward() # reverse-mode differentiation
        optimizer.step() # update weights and biases

    with torch.no_grad():
        validation_loss = loss_fn(model(X_val), y_val)
```

Three lines carry most of the apparent “machine learning”: forward evaluation, derivative evaluation, and a numerical optimization step. The scientific work lies around those lines: how the data were generated, what information is withheld, what quantity is predicted, which baseline is appropriate, and how error is interpreted.

7 From random test points to new physical systems

In ordinary supervised learning, observations are often shuffled and divided randomly into training and test sets. That procedure is useful when future observations are exchangeable with past observations. It can be misleading in physics when many observations come from the same trajectory, specimen, simulation, spatial field, or experimental run.

Suppose 60 time samples from one oscillator are randomly divided between training and test sets. A model can be tested on a time sample it has not seen while still having seen 59 neighboring samples from the same physical system. The test point is new; the system is not. For many scientific questions, the relevant prediction is instead a complete trajectory at a parameter pair absent from training.

A validation hierarchy for physics. For clarity, we use five descriptive levels throughout the article. A *sample split* withholds individual points from systems represented in training. A *system split* withholds complete trajectories, specimens, or simulation runs. *Parameter interpolation* withholds parameter combinations that remain inside the training envelope. *Parameter extrapolation* moves beyond that envelope. Finally, a *regime shift* changes the qualitative character of the dynamics, geometry, forcing, or constitutive behavior. These labels are operational rather than universal, but they force the evaluation question to be stated in physical terms.

The distinction is not unique to this oscillator. Scientific-ML benchmark efforts increasingly evaluate models across parameter, geometry, resolution, and distribution shifts rather than relying on a single random split.[14, 15] Distribution shift can also matter when learned components are embedded inside a simulation, where prediction errors may feed back into later states rather than remain passive output errors.[18] Studies of physics-informed models likewise show that behavior outside the training domain requires explicit investigation.[16]

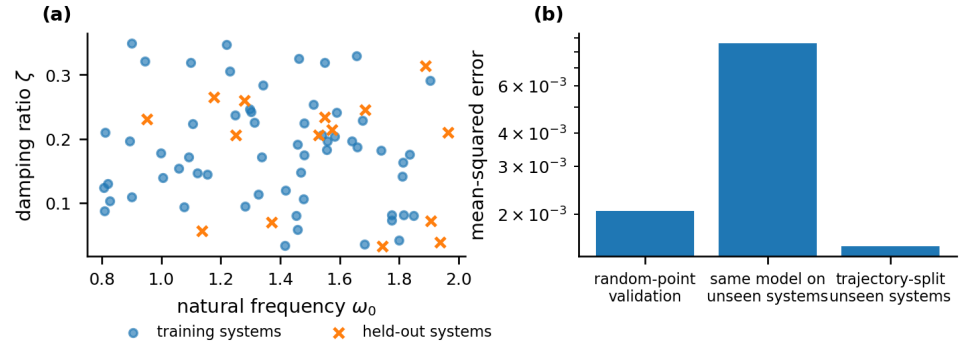


Figure 3. Why a validation split is part of the physical model assessment. (a) Training and held-out oscillator systems in the (ω_0, ζ) plane. (b) A random-point validation error can be much smaller than the error obtained when the same fitted model is applied to completely unseen systems.

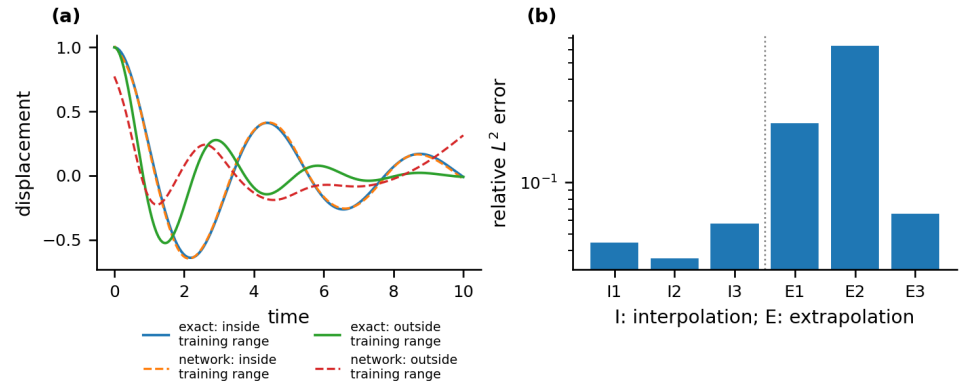


Figure 4. Interpolation and extrapolation are different scientific questions. (a) Example trajectories inside and outside the parameter range used for training. (b) Relative errors for three interpolation cases (I1–I3) and three extrapolation cases (E1–E3).

Our miniature experiment is shown in Fig. 3. A model fitted after a random point split among 60 source trajectories obtains random-point validation MSE $2.057\text{e-}03$. Without retraining, the same model is then evaluated on 15 entirely new trajectories and its error rises to $8.613\text{e-}03$, about a factor of four larger. A separately trained model using a trajectory-level protocol gives MSE $1.521\text{e-}03$ on its held-out trajectories. The exact numerical values are not universal; the lesson is that a test error only answers the question encoded by the split.

8 Interpolation, extrapolation, and the training envelope

A second distinction concerns the parameter domain. We evaluate three cases inside the sampled parameter rectangle and three outside it. For each case we compare the network prediction with Eq. (3) on a dense time grid. The relative error is

$$\epsilon_{\text{rel}} = \frac{\|x_\theta - x_{\text{exact}}\|_2}{\|x_{\text{exact}}\|_2}. \quad (12)$$

The mean error over the three interpolation cases is 0.046. For the three extrapolation cases it is 0.305. Figure 4 shows that this average conceals substantial case-to-case variation: one out-of-range case remains reasonable, while another deteriorates strongly.

It is tempting to summarize this result as “neural networks extrapolate poorly.” That statement is too crude. Extrapolation depends on the target function, parameterization, architecture, representation, and the direction in which the training support is left. The safer conclusion is operational: *extrapolation is not certified by interpolation accuracy*. If the scientific use case requires predictions at new damping ratios, frequencies, temperatures, Reynolds numbers, compositions, or geometries, those shifts should appear explicitly in the validation design.

Sensitivity to neural initialization. The representative plots above use a fixed seed so that every number can be reproduced exactly. Because neural training is nonconvex, we also repeated the

Table 2. Initialization robustness for the oscillator benchmark. Values are mean $\pm$ sample standard deviation over three independent neural-network initializations; the data-generating parameter sets are held fixed.

Quantity	Three-seed result
Random-point validation MSE	$1.98e-03 \pm 1.04e-03$
Same models on unseen trajectories, MSE	$8.39e-03 \pm 2.33e-03$
Parameter interpolation, mean relative L^2	0.062 ± 0.014
Parameter extrapolation, mean relative L^2	0.273 ± 0.028

oscillator experiment with three independent parameter initializations. Table 2 shows that the qualitative conclusion is not a single-seed accident. For models trained with a random-point split, the unseen-system error is on average 4.63 ± 1.22 times the random-point validation error. For the trajectory-level models, extrapolation error remains several times larger than interpolation error across all three initializations.

9 A familiar qualitative regime shift: simple pendulum

Parameter extrapolation is not the strongest possible test. A model may also be asked to cross a physical boundary at which the qualitative behavior of the system changes. A familiar example is an ideal simple pendulum of length ℓ ,

$$\ddot{\theta} + \frac{g}{\ell} \sin \theta = 0, \quad \theta(0) = 0, \quad \dot{\theta}(0) = \omega_0. \quad (13)$$

The specific mechanical energy is

$$E_0 = \frac{1}{2} \ell^2 \omega_0^2, \quad (14)$$

because the bob starts at the lowest point. The pendulum reaches the unstable upright position when its specific energy is

$$E_s = 2g\ell, \quad \omega_c = 2\sqrt{\frac{g}{\ell}}. \quad (15)$$

For the benchmark we set $\ell = 1$ m and plot the equivalent normalized energies $E_0/\ell^2 = \omega_0^2/2$ and $E_s/\ell^2 = 2g/\ell$. This separatrix is a clear physical boundary: for $\omega_0 < \omega_c$ the pendulum reverses direction and executes bounded oscillations, whereas for $\omega_0 > \omega_c$ it passes over the top and continues in rotation. No specialized PDE notation is needed to see that the qualitative motion has changed.

A small network $\theta_\theta(t, \omega_0)$ is trained only on 20 oscillatory systems with $0.8 \leq \omega_0 \leq 5.5$ rad s⁻¹. Six additional oscillatory systems are held out for ordinary system-level validation. A separate four-system set uses $6.7 \leq \omega_0 \leq 8.0$ rad s⁻¹, beyond the separatrix. The latter is not merely another random test set: the motion itself has changed from libration to continuous rotation.

Across three independent network initializations, mean relative L^2 error is 0.079 ± 0.006 on the held-out oscillatory systems and 1.080 ± 0.027 after crossing into the rotation regime, a roughly 13.7-fold increase. Figure 5 makes the failure visible. The network follows an unseen oscillatory trajectory reasonably closely, but on the rotation case it bends back toward the bounded behavior represented in the training set instead of continuing to accumulate angle.

This elementary example sharpens the hierarchy introduced in Sec. 7. A parameter can be numerically close to the training set while the associated physical behavior is qualitatively different. Conversely, a large parameter displacement need not create a new regime. If a stability threshold, phase boundary, bifurcation, onset criterion, or change of constitutive law is known, validation should be organized around that physical boundary rather than around Euclidean distance in parameter space alone. Recent scientific-ML studies make related points in substantially larger settings, including CFD extrapolation[17] and benchmarks spanning multiple physical regimes.[19]

10 Do we need deep learning for this problem?

An introductory article should not teach a neural method without teaching the baseline question. Equation (1) is easy to integrate directly, and the tabulated map in Eq. (4) can also be approximated by classical interpolation. We therefore compare the neural surrogate with a local Gaussian radial-basis-function (RBF) interpolant fitted to exactly the same training table, and with a high-accuracy numerical ODE solution for the forward problem.

The RBF interpolant obtains mean relative interpolation error 0.049, close to the neural value 0.046. Its mean extrapolation error rises to 1.793, even larger than the neural surrogate's already substantial 0.305. Thus the neural network is competitive with this particular data-driven baseline

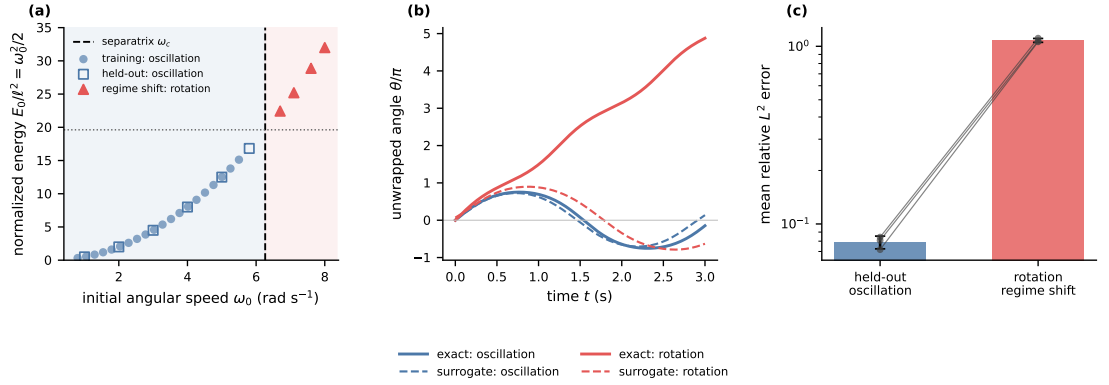


Figure 5. A familiar qualitative regime shift in a simple pendulum. (a) Normalized initial energy $E_0/l^2 = \omega_0^2/2$ as a function of initial angular speed. The blue shaded region corresponds to bounded oscillation, whereas the red shaded region corresponds to continuous rotation; the dashed line marks the separatrix ω_c , and the horizontal dotted line marks its normalized energy E_s/l^2 . Filled circles are training systems, open squares are held-out oscillatory systems, and triangles are rotation-regime systems. (b) Exact and surrogate unwrapped angles for a representative held-out oscillation and a representative rotation; the two-row legend is placed below the panels. The surrogate follows the bounded motion much better than the unseen continuous rotation. (c) Mean relative L^2 error for the two test groups across three neural initializations; bars show the mean and sample standard deviation, and connected points show individual seeds.

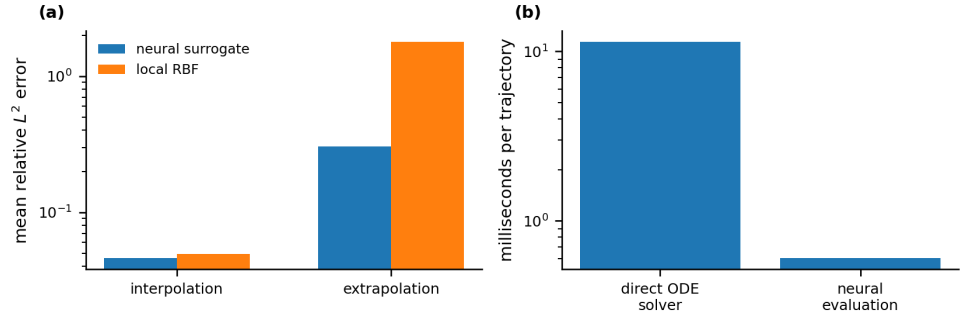


Figure 6. Conventional baselines change the interpretation of the neural result. (a) The neural surrogate and a local RBF approximation are similar on the selected interpolation cases; both degrade outside the training envelope, with the RBF degrading more severely here. (b) Neural evaluation is faster than a high-accuracy direct ODE solve only after the up-front training cost has been paid.

inside the training domain and less fragile on the selected extrapolation cases, but neither result licenses general claims about extrapolation.

The direct ODE solver is a different baseline because it uses the governing equation rather than learning from the precomputed table. Its mean relative error against the analytic solution is approximately $5.52\text{e-}11$. On the reference CPU, one 301-point trajectory takes about 11.40 ms, while a neural forward evaluation takes about 0.60 ms after training. The network required roughly 11.9 s of training. These timings are hardware- and implementation-dependent, but they illustrate an important rule: for a single forward solve of a known low-dimensional model, the conventional solver is the natural choice. A surrogate becomes interesting when its training cost can be amortized across many queries or embedded inside a larger task.

This baseline exercise is intentionally modest. RBF interpolation is not the optimal classical method for every parameterized dynamical system, and the oscillator is too simple to motivate a deep surrogate on performance grounds. That is precisely why it is pedagogically useful: a student can see that the correct comparison is not “neural network versus no model,” but “neural network versus the strongest simple method appropriate to the scientific task.”

11 A small inverse problem

The same differentiability that supports training can be used to infer physical parameters. Suppose noisy displacement measurements y_j are available at times t_j , but ω_0 and ζ are unknown. Once a

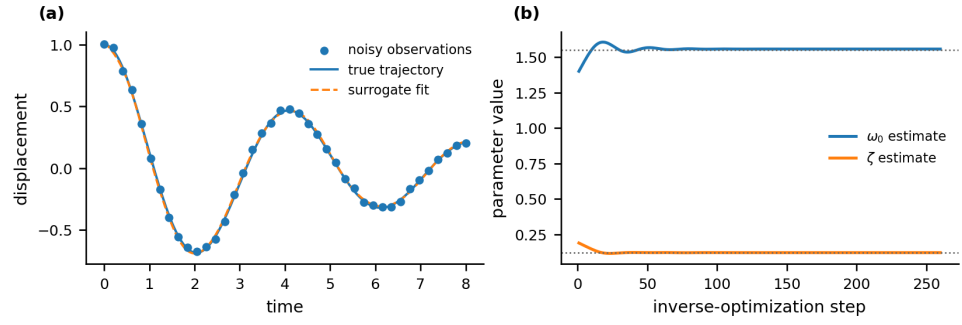


Figure 7. Inverse use of the differentiable surrogate. (a) Noisy displacement observations and the trajectory corresponding to the recovered parameters. (b) Optimization history for ω_0 and ζ ; dotted horizontal lines mark the true values.

surrogate $f_\theta(t, \omega_0, \zeta)$ has been trained, define

$$\mathcal{J}(\omega_0, \zeta) = \frac{1}{M} \sum_{j=1}^M [f_\theta(t_j, \omega_0, \zeta) - y_j]^2. \quad (16)$$

We now keep θ fixed and optimize the physical parameters instead. Automatic differentiation provides $\partial\mathcal{J}/\partial\omega_0$ and $\partial\mathcal{J}/\partial\zeta$ through the network.

In the demonstration, 40 observations are generated from $(\omega_0, \zeta) = (1.55, 0.12)$ with additive Gaussian noise of standard deviation 0.015. The recovered values are (1.5566, 0.1223). Figure 7 shows the fitted trajectory and parameter-optimization history.

This is not offered as a replacement for established parameter-estimation methods. For Eq. (1), fitting the exact or numerical solution directly is straightforward and preferable. The example is included because it changes the reader's mental model of a surrogate: it is not only a fast map from parameters to observations; it is also a differentiable component through which sensitivities can be propagated. In larger computational pipelines, that property can be valuable for optimization, calibration, and inverse design.

12 Teaching applications and practical lessons

The oscillator experiment is small enough that its main lessons can be stated plainly.

1. **Define the prediction unit before the train/test split.** If future use concerns new specimens, trajectories, geometries, or experiments, hold out those units rather than random points within them.
2. **Separate interpolation from extrapolation.** A model that is accurate inside the sampled parameter envelope has not thereby been validated outside it.
3. **Identify qualitative regime boundaries when they are known.** Stability thresholds, phase changes, bifurcations, and constitutive transitions can be more scientifically meaningful than geometric distance in parameter space.
4. **Use a physical or numerical baseline.** If a governing-equation solver is cheap and accurate, deep learning may add no value for the forward task.
5. **Use a non-neural data-driven baseline.** A simple interpolant, reduced model, Gaussian process, or regression method can reveal whether network complexity is actually buying accuracy.
6. **Report the data-generation cost.** A surrogate trained on expensive simulations inherits their computational cost even if inference is fast.
7. **Preserve units and scales conceptually even when inputs are normalized numerically.** Scaling should aid optimization without erasing physical interpretation.
8. **Treat reproducibility as part of the result.** Seeds, parameter ranges, split definitions, software versions, raw predictions, and training histories should be retained, not only the final plot.

12.1 Suggested student activities

The review supports short activities that make the validation issue observable rather than merely verbal. Students can (i) retrain the oscillator model after replacing a random point split with a trajectory-level split; (ii) plot interpolation and extrapolation errors against the training envelope; (iii) move the pendulum initial speed across the exact separatrix $\omega_c = 2\sqrt{g/\ell}$ and explain the change from bounded oscillation to continuous rotation; (iv) reproduce the RBF and direct-solver baselines; and (v) use the differentiable surrogate to recover noisy oscillator parameters before comparing the result with direct fitting. A suitable assessment asks students to state precisely what was withheld, identify the baseline used, and separate an empirical observation from a general claim.

These activities also provide natural opportunities to discuss limitations. A good student report should note that the examples are low-dimensional and analytically controlled, that the selected architecture is not universally optimal, and that reproducibility of the numerical result does not by itself establish transfer to a new physical domain.

13 Boundaries of the review

The restraint of scope is deliberate. Convolutional and attention-based architectures, graph neural networks, equivariant networks, generative models, and reinforcement learning are not needed to understand the computational ideas developed here. Neither do we cover physics-informed neural networks, which place differential-equation residuals in the training objective, or neural operators, which learn maps between functions. Those topics raise additional questions about physical constraints, discretization dependence, operator approximation, and scientific validation.^[20, 14]

The present article should therefore be read as a first layer rather than a survey. Its central message is that the network architecture is only one component of a scientific-learning problem. The definition of the prediction task and the validation regime are equally important. A student who can state precisely what was withheld from training, what baseline was beaten, and where the model is being asked to extrapolate is better prepared for more advanced scientific machine learning than one who has simply implemented a larger network.

14 Conclusions and educational takeaways

We have introduced deep feed-forward neural networks through a single parameterized physical system. This allows the mathematical and computational ideas—nonlinear approximation, numerical optimization, back propagation, automatic differentiation, and surrogate evaluation—to be developed without changing applications every few pages. More importantly, it permits a controlled discussion of scientific generalization.

The numerical results make four points. First, a random-point validation error can substantially understate the error obtained when the same fitted model is applied to new physical systems, and the gap persists across independent neural initializations. Second, interpolation accuracy does not certify parameter extrapolation. Third, the simple-pendulum example shows that a qualitative regime boundary can be more informative than parameter distance: a network accurate on unseen oscillatory systems can fail abruptly after the exact energy separatrix is crossed into continuous rotation. Fourth, the value of a neural surrogate is visible only relative to alternatives: on the chosen oscillator interpolation cases it performs similarly to a local RBF interpolant, while the direct ODE solver remains the correct tool for an isolated forward solve. The differentiable surrogate nevertheless provides a convenient bridge to inverse inference.

None of these observations should be read as universal rankings of methods, nor are random-versus-extrapolatory splits themselves new. The contribution is the way these ideas are assembled into a reproducible introductory workflow. For physicists, “Does the model generalize?” should almost always be followed by “to which samples, systems, parameters, and qualitative regimes?” That question becomes more, not less, important as scientific machine-learning models grow in scale and apparent capability.

Acknowledgments

No specific acknowledgments.

Funding

No specific grant from any funding agency in the public, commercial, or not-for-profit sectors supported this work.

Author contributions

Othman H. Y. Zalloum conceived the review, performed the numerical analyses, prepared the code and figures, and wrote the manuscript.

Data availability

The package accompanying this manuscript contains the fixed-seed and multi-initialization Python scripts, generated CSV files for both physical examples, trained state dictionaries, publication figures, and the Jupyter notebook. These materials are intended to serve as supplementary teaching and reproducibility resources and can be deposited in a permanent repository with a DOI upon publication.

Supplementary data

The supplementary package includes the Jupyter notebook, source code, data tables, model state files, and figures used in the review.

Conflict of interest

The author declares no conflict of interest.

References

- [1] C. F. Higham and D. J. Higham, “Deep Learning: An Introduction for Applied Mathematicians,” *SIAM Review* **61**, 860–891 (2019), doi:10.1137/18M1165748.
- [2] P. Mehta, C.-H. Wang, A. G. R. Day, C. Richardson, M. Bukov, C. K. Fisher, and D. J. Schwab, “A high-bias, low-variance introduction to Machine Learning for physicists,” *Physics Reports* **810**, 1–124 (2019), doi:10.1016/j.physrep.2019.03.001.
- [3] G. Carleo *et al.*, “Machine learning and the physical sciences,” *Reviews of Modern Physics* **91**, 045002 (2019), doi:10.1103/RevModPhys.91.045002.
- [4] D. Ray, O. Pinti, and A. A. Oberai, *Deep Learning and Computational Physics* (Springer, Cham, 2024), doi:10.1007/978-3-031-59345-1.
- [5] G. Café de Miranda, G. G. de Lima, and T. de S. Farias, “An introduction to Neural Networks for Physicists,” arXiv:2505.13042 (2025).
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, Cambridge, MA, 2016).
- [7] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Math. Control Signals Syst.* **2**, 303–314 (1989).
- [8] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks* **2**, 359–366 (1989).
- [9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature* **323**, 533–536 (1986).
- [10] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. (Springer, New York, 2006).
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” International Conference on Learning Representations (2015).
- [12] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: a survey,” *J. Mach. Learn. Res.* **18**, 1–43 (2018).
- [13] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems* **32** (2019).
- [14] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, “PDEBench: An extensive benchmark for scientific machine learning,” *Advances in Neural Information Processing Systems* **35** (2022).
- [15] S. Künzli, F. Grötschla, J. Mathys, and R. Wattenhofer, “SURF: A generalization benchmark for GNNs predicting fluid dynamics,” *Proceedings of Machine Learning Research* **231**, 13:1–13:23 (2024).
- [16] A. Bonfanti, R. Santana, M. Ellero, and B. Gholami, “On the generalization of PINNs outside the training domain and the hyperparameters influencing it,” *Neural Computing and Applications* **36**, 22677–22696 (2024), doi:10.1007/s00521-024-10178-2.
- [17] A. Rabeh, E. Herron, A. Balu, S. Sarkar, C. Hegde, A. Krishnamurthy, and B. Ganapathysubramanian, “Benchmarking scientific machine-learning approaches for flow prediction around complex geometries,” *Communications Engineering* **4**, 182 (2025), doi:10.1038/s44172-025-00513-3.
- [18] J. Zhao and Q. Li, “Mitigating Distribution Shift in Machine Learning–Augmented Hybrid Simulation,” *SIAM Journal on Scientific Computing* **47**, C475–C500 (2025), doi:10.1137/23M1615425.

- [19] M. Chu, Y. Liu, A. Biswas, and H.-W. Shen, “Do Physics Foundation Models Learn Generalizable Physics? A Bias-Aware Benchmark Across Physical Regimes and Distribution Shifts,” arXiv:2605.29283 (2026).
- [20] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics* **3**, 422–440 (2021), doi:10.1038/s42254-021-00314-5.